

NAME : S. BHAVYA

COURSE CODE : CSA0313

REG NO : 192524376

COURSE COURSE: DATA STRUCTURE

16/8/26

CO4: AT1 - COMPARATIVE ANALYSIS

1. Introduction:

Hashing is a technique used to store & retrieve data efficiently.

When 2 keys produce the same hash value, a collision occurs.

Linear Probing and Separate Chaining are 2 methods used to resolve collisions.

Comparison Table:

Basis	Linear Probing	Separate Chaining
Collision Handling	Searches the next empty slot.	Stores elements in a linked list at same index.
Clustering	High Primary Clustering	Very little Clustering.
Memory Usage	Less Memory	More memory.
High Load Factor	Performance Decreases greatly.	Handles high load factors better.
Deletion	Difficult	Easy
Cache Performance	Better	Comparatively lower
Large-Scale Use	Suitable for low load	Suitable for high load
Search (average)	$O(1)$	$O(1 + \alpha)$
Insert (average)	$O(1)$	$O(1)$
Worst Case Search	$O(n)$	$O(n)$

a) Clustering Effect:

Linear Probing:

When a collision occurs, the algorithm checks the next position until an empty slot is found.

Eg:

Index :	0	1	2	3	4	5
Data :	10	20	30	40	-	-

The occupied positions become consecutive, creating primary clustering.

Separate Chaining:

Each table index contains a linked list

0 → 10 → 30 → 50

1 → 21

2 → 12 → 42

Collisions are stored in the same bucket, so primary clustering is avoided.

b) Memory Usage:

Linear Probing:

- Stores all elements directly in the hash table
- Requires no extra pointers
- Uses less memory.

Separate Chaining:

- Each collided element requires a linked-list node.
- Additional memory is needed for pointers.
- Therefore, memory overhead is higher.

c) Performance Under High Load Factor:

Load factor:

$$\alpha = \frac{\text{No. of Elements}}{\text{Table Size}}$$

Linear Probing:

- More collisions occur.
- Clusters become larger.
- Searching & Insertion require many probes

Separate Chaining:

- Multiple Elements are stored in 1 bucket.
- Performance decreases gradually.
- It can even work when the load factor is greater than 1.

d) Separate Chaining is more suitable

Reason:

- Handles high load factors efficiently
- Easier insertion & deletion.
- Less affected by clustering.
- Better scalability for large datasets.

Conclusion:

Separate Chaining is generally the preferred choice for large-scale applications due to its scalability, flexibility & stable performance.

2. Introduction:

Sorting a linked list containing 10 million nodes requires an algorithm with good complexity, low memory overhead, and efficient handling of linked-list nodes.

The possible algorithms are Quick Sort and Merge Sort.

Comparison Table:

Factor	Quick Sort	Merge Sort
Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Space	$O(\log n)$ average	$O(\log n)$ recursion stack
Stability	Usually not stable	Stable
Linked-List suitability	Moderate	Excellent
Pivot Required	Yes	No
Large dataset	Risk of Worst Case	More Predictable

a) Time Complexity:

→ Quick Sort normally takes $O(n \log n)$ but can become $O(n^2)$ if poor pivots are selected.

→ Merge Sort always provides $O(n \log n)$ performance, making it more reliable for 10 million nodes.

b) Memory Requirement:

- Quick Sort normally uses recursion stack space. In the worst case, its stack can become $O(n)$.
- Merge Sort divides the list & merges the nodes by changing pointers, therefore, it doesn't need to copy all 10 million nodes into another list.

c) Merge Sort is Better for Linked Lists:

Merge Sort is well suited to linked lists because nodes can be rearranged using pointers.

Eg:

List 1: $10 \rightarrow 30 \rightarrow 50$

List 2: $20 \rightarrow 40 \rightarrow 60$

Merged: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$

d) Stability:

- Merge sort is stable, so equal elements maintain their original order.
- This is useful when sorting student records, employee records or database records.

e) If the Data Were Arrays:

- For arrays, Quick Sort can be more attractive because it has good cache performance & can be implemented with very little extra memory.
- Merge Sort is preferred when stability or guaranteed $O(n \log n)$ performance is required.

f) Conclusion:

- For 10-million linked-list nodes, Merge Sort is preferable because it provides guaranteed $O(n \log n)$ time, efficient pointer-based merging, stability & predictable performance.

3. Introduction:

→ Quadratic Probing & Double Hashing are open-addressing techniques used to resolve collisions in hash tables.

→ Both try to find another empty position when 2 keys produce the same hash index.

Comparison Table:

Factor	Quadratic Probing	Double Hashing
Probe Formula	$h(k) + i^2$	$h_1(k) + i \times h_2(k)$
Primary Clustering	Reduced	Almost Eliminated
Secondary Clustering	Present	Very Low
Complexity - Average	$O(1)$	$O(1)$
Complexity - Worst	$O(n)$	$O(n)$
Implementation	Easier	More Complex
Performance	Good	Generally better
Dense Tables	Less suitable	More suitable

a) Clustering Issues:

→ In Quadratic Probing, after a collision, positions are checked using quadratic increments:

$$* h(k), h(k) + 1^2, h(k) + 2^2, h(k) + 3^2, \dots$$

→ Double Hashing, two hash functions are used:

$$* h(k, i) = [h_1(k) + i \times h_2(k)] \text{ mod } m.$$

b) Collision Resolution Efficiency:

→ Quadratic probing checks positions using increasing square values:

$$h, h+1, h+4, h+9, \dots$$

→ Double Hashing uses:

$$h, h+d, h+2d, h+3d, \dots$$

where d is obtained from the second hash function.

c) Dense Hash Tables:

→ When the load factor is high, many positions in the table are already occupied.

→ Quadratic probing may require many probes & may visit every table position depending on the table size & probing formula.

→ Double hashing generally performs better for dense tables because it provides a more varied probe sequence.

Conclusion:

Quadratic Probing is simple & effective for moderately loaded tables, while Double Hashing provides better collision handling & is more suitable for dense hash tables.

4. Introduction:

The best-case complexity represents the minimum time taken by a sorting algorithm, while the worst-case complexity represents the maximum time required for a given input.

Comparison Table:

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

1. Best-Case Complexity:

→ Best case occurs when the input is favorable.

Eg: Insertion Sort takes $O(n)$ when the elements are already sorted because very few movements are required.

2. Worst-Case Complexity:

→ Worst-case occurs when the input causes maximum comparisons or movements.

Eg: Quick Sort can become $O(n^2)$ when a poor pivot is repeatedly selected.

3. Importance in Algorithm Selection:

Best and worst cases help us choose the appropriate algorithm:

- * **Nearly Sorted Data**: Insertion Sort
- * **Guaranteed Performance**: Merge Sort or Heap Sort
- * **General - purpose sorting**: Quick Sort
- * **Small datasets**: Insertion Sort

Conclusion:

- Best - Case complexity shows how efficiently an algorithm can perform under favourable conditions.
- Worst - Case complexity shows its maximum cost.
- For large datasets, algorithms with $O(n \log n)$ worst case complexity, such as Merge Sort & Heap Sort are generally more reliable.

5. Introduction:

- Quick Sort can be implemented using recursion or an iterative approach. It is a divide & conquer sorting algorithm.
- Both methods have similar sorting logic, but they differ mainly in how they manage memory & stack operations.

Comparison Table:

Factor	Recursive Quick Sort	Iterative Quick Sort
Implementation	Simple	More Complex
Stack	System call, back	Explicit / Programmer controlled stack.
Average Time	$O(n \log n)$	$O(n \log n)$
Worst Time	$O(n^2)$	$O(n^2)$
Average Space	$O(\log n)$	$O(\log n)$
Worst Space	$O(n)$	$O(n)$
Stack Overflow	Higher risk	Lower Risk
Memory Control	Limited	Better

1. Space Complexity

→ In Recursive Quick Sort, every recursive call is stored in the system call stack. With balanced partitions, the recursion depth is:

$$O(\log n)$$

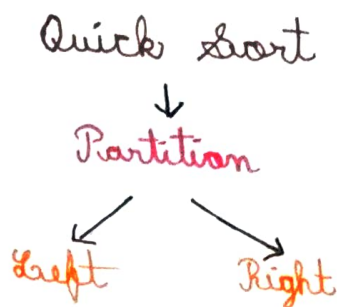
→ However, if the pivot repeatedly produces unbalanced partitions, the depth can become:

$$O(n)$$

→ This increases the risk of stack overflow.

2. Ease of Implementation:

→ Recursive Quick sort, recursion is replaced because the algorithm naturally follows the divide & conquer structure:



→ Iterative Quick sort requires creating & managing a stack manually, so the code is slightly more complicated.

3. Stack Overflow Risk:

→ Recursive Quick Sort can cause overflow if partitions become highly unbalanced especially for already sorted data with poor pivot selection.

→ Iterative Quick Sort avoids excessive use of the system call stack because it manages the stack explicitly.

Conclusion:

→ Recursive Quick Sort is simpler & easier to understand.

→ Iterative Quick Sort provides better control over stack memory.

→ For large datasets where stack overflow is a concern, the iterative approach can be preferred.